

Semantic Analysis

1. Introduction to Semantic Analysis

1.1 What is Semantic Analysis?

Definition: Semantic Analysis is the third phase of a compiler that checks whether the program is **meaningful** according to the rules of the programming language.

Simple Explanation: While syntax analysis checks if code is grammatically correct, semantic analysis checks if it makes logical sense.

Alternative Term: Also called **Semantic Checking** or **Context-Sensitive Analysis**.

1.2 Position in Compiler Phases

Complete Compilation Flow:

SOURCE CODE

↓

[1] LEXICAL ANALYSIS (Scanner)

↓ produces tokens

[2] SYNTAX ANALYSIS (Parser)

↓ produces parse tree

[3] SEMANTIC ANALYSIS ← WE ARE HERE

↓ produces annotated tree

[4] INTERMEDIATE CODE GENERATION

↓

[5] CODE OPTIMIZATION

↓

[6] CODE GENERATION

↓

TARGET CODE (Machine Code)

What Happens Before Semantic Analysis:

- Lexical Analysis: Breaks code into tokens
- Syntax Analysis: Checks grammatical structure

What Happens After Semantic Analysis:

- Intermediate Code Generation: Creates intermediate representation
- Optimization: Improves code efficiency
- Code Generation: Produces machine code

1.3 Purpose of Semantic Analysis

Main Goal: Ensure the program is meaningful and follows the semantic rules of the language.

Key Functions:

1. Verify consistency of declarations and usage
2. Check type compatibility
3. Ensure proper scope of variables
4. Validate function calls
5. Detect logical errors

Important Note: Semantic errors occur in programs that are syntactically correct but logically invalid.

2. Syntax Error vs Semantic Error

2.1 Syntax Errors

Definition: Violations of the **grammar rules** of the language.

Detected By: Parser (Syntax Analyzer)

Examples:

ERROR 1: Missing semicolon

```
int x = 5
```

Syntax Error: Expected ';' after statement

ERROR 2: Incorrect operator placement

```
int y = + 5 * ;
```

Syntax Error: Unexpected operator

ERROR 3: Unbalanced parentheses

```
if (x > 5 {  
    y = 10; }
```

Syntax Error: Expected ')' before '{'

ERROR 4: Wrong keyword order

then if condition statement

Syntax Error: Unexpected keyword 'then'

2.2 Semantic Errors

Definition: Violations of the **meaning or logic** rules of the language.

Detected By: Semantic Analyzer

Key Point: Program is syntactically correct but doesn't make logical sense.

Examples:

ERROR 1: Type mismatch

```
int x = 5;
```

```
x = "hello";
```

Semantic Error: Cannot assign string to integer variable

ERROR 2: Undeclared variable

```
y = x + 10;
```

Semantic Error: Variable 'y' used before declaration

ERROR 3: Incompatible operation

```
int a = 5;
```

```
int b = a / "text";
```

Semantic Error: Cannot divide integer by string

ERROR 4: Wrong number of arguments

```
int add(int x, int y) {
```

```
    return x + y;
```

```
}
```

```
int result = add(5);
```

Semantic Error: Function 'add' expects 2 arguments, got 1

2.3 Comparison Table

ASPECT	SYNTAX ERROR	SEMANTIC ERROR
Violates	Grammar rules	Meaning / logic rules
Detected By	Parser	Semantic Analyzer
Compiler Phase	Phase 2 (Syntax Analysis)	Phase 3 (Semantic Analysis)
Program Structure	Invalid structure	Valid structure
Code Meaning	May or may not be clear	Clear but illogical
Example 1	<pre>if x > 5 then</pre> (missing condition)	<pre>int x = 5;</pre> <pre>x = "hello";</pre> (type mismatch)
Example 2	<pre>int x = ;</pre> (missing value)	<pre>int x;</pre> <pre>y = x + 10;</pre> (y undeclared)
Example 3	<pre>x + * y</pre> (wrong operator order)	<pre>int x = 5;</pre> <pre>bool b = x + "text";</pre> (invalid operation)

3. Main Tasks of Semantic Analysis

3.1 Overview of Tasks

Four Primary Responsibilities:

TASK 1: Type Checking

TASK 2: Symbol Table Management

TASK 3: Scope Checking

TASK 4: Function and Procedure Checking

3.2 Task 1: Type Checking

Definition: Ensures that operands in expressions are of compatible types.

Purpose: Detect type mismatch errors at compile time.

What is Checked:

CHECK 1: Variable declarations match their usage

CHECK 2: Operators are applied to compatible types

CHECK 3: Assignments are type-compatible

CHECK 4: Function return types match declarations

CHECK 5: Array indices are integers

3.3 Task 2: Symbol Table Management

Definition: A symbol table is a data structure used by a compiler to store information about identifiers (variables, functions, arrays, classes, etc.) that appear in a program.

Maintaining a data structure (symbol table) that stores information about all identifiers in the program.

Attributes Stored in a Symbol Table

Each entry in a symbol table typically contains:

- **Name** – Identifier name
- **Type** – Data type (int, float, char, function return type, etc.)
- **Scope** – Global, local, block-level
- **Memory Location / Address**
- **Size** – Memory required
- **Value** – Current value (optional)
- **Line Number(s)** – Where declared/used
- **Parameter List** – For functions (number and type of parameters)
- **Return Type** – For functions

Symbol Table Example:

SOURCE CODE:

```
int x = 10;

float y = 3.14;

void foo(int a, int b) {
    int z = a + b;
}
```

Operations on Symbol Table:

OPERATION 1: INSERT

- Add new identifier to table
- Done during declaration processing

OPERATION 2: LOOKUP

- Search for identifier in table
- Done during usage checking

OPERATION 3: UPDATE

- Modify information about identifier
- Done when type or value changes

OPERATION 4: DELETE

- Remove identifier from table
- Done when leaving scope

Example Usage:

CODE:

```
int x = 5;      // INSERT: Add 'x' to symbol table
x = x + 10;     // LOOKUP: Find 'x', verify it exists and is int
if (x > 10) {
    int y = 20; // INSERT: Add 'y' to symbol table (local scope)
}              // DELETE: Remove 'y' from table (leaving scope)
```

Significance / Importance of Symbol Table

1. Semantic Analysis

- Checks whether variables are declared before use
- Detects duplicate declarations
- Enforces scope rules

2. Type Checking

- Ensures type compatibility in expressions and assignments

3. Memory Allocation

- Helps allocate and manage memory for variables

4. Code Generation

- Provides address, size, and other details needed to generate target code

5. Error Detection

- Identifies undeclared identifiers, multiple definitions, and scope violations

3.4 Task 3: Scope Checking

Definition: Ensures variables are used only within their valid scope (visibility region).

Types of Scope:

SCOPE TYPE 1: GLOBAL SCOPE

- Variables declared outside all functions
- Visible throughout entire program
- Lives for entire program execution

SCOPE TYPE 2: LOCAL SCOPE (Function Scope)

- Variables declared inside a function
- Visible only within that function
- Lives only during function execution

SCOPE TYPE 3: BLOCK SCOPE

- Variables declared inside a block { }
- Visible only within that block
- Lives only within block execution

{

int x = 10; // block-scoped variable

printf("%d", x); // valid

}

3.5 Task 4: Function and Procedure Checking

Definition: Validates that function calls match function declarations.

What is Checked:

CHECK 1: NUMBER OF PARAMETERS

- Call must provide correct number of arguments

CHECK 2: PARAMETER TYPES

- Argument types must match parameter types

CHECK 3: RETURN TYPE

- Function must return correct type
- Return statement must be present (for non-void functions)

CHECK 4: FUNCTION EXISTENCE

- Called function must be declared

5. Type Checking in Detail

5.1 Static vs Dynamic Type Checking

STATIC TYPE CHECKING:

Definition: Types are checked at COMPILE TIME

Used In: C, C++, Java, C#, Rust, Go

ADVANTAGES:

- ✓ Errors caught before program runs
- ✓ Better performance (no runtime checks)
- ✓ Code is safer and more reliable
- ✓ Better tooling support (IDEs can help)

DISADVANTAGES:

- ✗ Less flexible
- ✗ More verbose code
- ✗ Longer development time

Example (Java):

```
int x = 5;
```

```
x = "hello";    // ✗ ERROR at compile time
```

DYNAMIC TYPE CHECKING:

Definition: Types are checked at RUNTIME

Used In: Python, JavaScript, Ruby, PHP

ADVANTAGES:

- ✓ More flexible
- ✓ Shorter code
- ✓ Faster prototyping
- ✓ Duck typing possible

DISADVANTAGES:

- ✗ Errors found at runtime
- ✗ Performance overhead
- ✗ Less safe
- ✗ Harder to maintain large codebases

Example (Python):

```
x = 5          # x is int
```

```
x = "hello"    # ✓ OK, x is now string (checked at runtime)
```

6. Common Semantic Errors

6.1 Error Category 1: Undeclared Variable Usage

Error: Using a variable that hasn't been declared.

EXAMPLE 1:

```
y = x + 10;
```

Semantic Error: Variable 'y' not declared

6.2 Error Category 2: Multiple Declarations

Error: Declaring the same variable more than once in the same scope.

EXAMPLE 1:

```
int x = 5;
```

```
int x = 10;
```

Semantic Error: Variable 'x' already declared

6.3 Error Category 3: Type Mismatch in Expressions

Error: Using incompatible types in operations.

EXAMPLE 1:

```
int x = 5;  
string y = "hello";  
int z = x + y;
```

Semantic Error: Cannot add int and string

6.4 Error Category 4: Wrong Function Calls

Error: Calling functions incorrectly.

EXAMPLE 1: Wrong number of arguments

```
int add(int a, int b) { return a + b; }  
int result = add(5);
```

Semantic Error: Expected 2 arguments, got 1

7. Control Statement Checking

- Conditions must evaluate to boolean
- Checks validity of break and continue
- Ensures return statement correctness